SAECOIN — NETWORK PARAMETERS SPECIFICATION

Document: Network_Parameters.md
Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Status: Draft / Planning
Consensus: Proof of Work (PoW)
Transaction Model: UTXO
Primary Development OS: Windows 11

---

1. PURPOSE

This document defines the proposed network-level parameters for the SAECOIN blockchain.

It covers:

- Mainnet and testnet identity
- P2P networking
- Network ports
- Block production timing
- Difficulty adjustment
- Block size and weight
- Transaction limits
- Peer management
- Synchronization
- Network upgrade compatibility

All parameters marked PROVISIONAL must be reviewed and finalized before implementation of consensus-critical functionality.

---

2. NETWORK TYPES

SAECOIN will support three separate network environments.

Network| Purpose| Status
Development| Local development and testing| Planned
Testnet| Public testing| Planned
Mainnet| Production blockchain| Not launched

Each network must have a unique identity.

Networks must not accidentally accept one another's blocks or transactions.

---

3. NETWORK IDENTIFICATION

Each network should have:

- Unique network magic bytes
- Unique chain identifier
- Unique genesis block hash
- Unique P2P configuration
- Separate data directory
- Separate wallet network configuration

Proposed identifiers:

Parameter	Development	Testnet	Mainnet
Network name	SAECOIN Development	SAECOIN Testnet	SAECOIN Mainnet
Network ID	DEV	TEST	MAIN
Genesis block	Separate	Separate	Separate
Data directory	Separate	Separate	Separate

The actual numeric or binary identifiers remain TO BE DECIDED.

Network identity must be validated before accepting peer data.

---

4. P2P NETWORKING

SAECOIN nodes will communicate through a peer-to-peer network.

Expected P2P capabilities:

- Peer discovery
- Peer connection
- Peer authentication through protocol checks
- Block announcements
- Block downloads
- Transaction announcements
- Transaction propagation
- Peer scoring
- Connection timeouts
- Misbehavior detection
- Peer disconnection

The P2P protocol must not trust information supplied by another peer without validation.

---

5. NETWORK PORTS

The following ports are proposed for planning purposes only.

Service	Proposed Port	Status
Mainnet P2P	28444	Provisional
Testnet P2P	38444	Provisional
Development P2P	48444	Provisional
Local REST API	28445	Provisional
Testnet REST API	38445	Provisional

Ports must be checked for conflicts and finalized before public deployment.

The REST API should not be exposed publicly by default.

Node operators should be able to configure ports.

---

6. BLOCK PRODUCTION PARAMETERS

Initial proposed parameters:

Parameter	Proposed Value
Target block interval	600 seconds
Target block interval in minutes	10 minutes
Initial block reward	50 SAC
Halving interval	210,000 blocks
Maximum supply target	21,000,000 SAC
Premine	0 SAC
Decimal precision	8

All values are provisional.

The issuance schedule must be mathematically verified using integer arithmetic.

The selected initial reward and halving interval must not be assumed to produce exactly 21 million SAC without verification.

---

7. DIFFICULTY ADJUSTMENT

Proof of Work difficulty must adjust as network mining power changes.

Objectives:

- Maintain the target block interval.
- Prevent extreme difficulty manipulation.
- Handle sudden changes in hashrate.
- Limit timestamp manipulation.
- Maintain predictable chain behavior.

The following details remain undecided:

- Difficulty adjustment algorithm
- Retarget interval
- Target timespan
- Minimum difficulty rules
- Maximum adjustment factor
- Timestamp validation rules
- Protection against difficulty manipulation

The algorithm must be simulated and tested before consensus implementation.

All nodes must calculate difficulty using identical deterministic rules.

---

8. BLOCK SIZE AND WEIGHT

Maximum block size has not been finalized.

The design must consider:

- Network bandwidth
- Block propagation time
- Node storage
- Validation performance
- Transaction throughput
- CPU and RAM requirements
- Decentralization

A block limit must prevent oversized blocks from exhausting node resources.

The final limit should be based on benchmarking rather than an arbitrary value.

---

9. TRANSACTION PARAMETERS

The following transaction-level limits require decisions:

Parameter| Status
Minimum transaction size| To be decided

Maximum transaction size| To be decided
Maximum inputs per transaction| To be decided
Maximum outputs per transaction| To be decided
Minimum transaction fee| To be decided
Fee calculation method| To be decided
Dust threshold| To be decided
Mempool size limit| To be decided
Transaction expiry policy| To be decided

Transaction serialization must be deterministic.

All nodes must calculate transaction identifiers consistently.

---

10. MEMPOOL POLICY

The mempool stores valid unconfirmed transactions.

Expected behavior:

- Validate incoming transactions.
- Reject invalid transactions.
- Check UTXO availability.
- Prevent conflicting spends.
- Apply fee policy.
- Limit memory usage.
- Remove confirmed transactions.
- Remove expired or invalid transactions according to policy.

Mempool policy may differ between nodes.

However, consensus validation rules must remain identical across all compatible nodes.

---

11. PEER CONNECTION LIMITS

Peer connection limits are not finalized.

The design must support:

- Maximum inbound connections
- Maximum outbound connections
- Connection timeout
- Handshake timeout
- Peer banning duration
- Duplicate connection handling

- Peer discovery limits

The defaults should be benchmarked on modest hardware.

Node operators should be able to configure connection limits.

---

12. BLOCK PROPAGATION

The network should propagate newly mined blocks efficiently.

Expected process:

1. Miner discovers a valid block.
2. Miner submits the block to its node.
3. Node validates the block.
4. Node updates its local chain state.
5. Node announces the block to peers.
6. Peers request and validate the block.
7. Valid peers update their local chain state.
8. Peers propagate the block onward.

Invalid blocks must never be accepted simply because they were announced by a trusted or well-connected peer.

---

13. BLOCKCHAIN SYNCHRONIZATION

New nodes must be able to synchronize from existing peers.

Synchronization must verify:

- Block header structure
- Proof of Work
- Previous block linkage
- Difficulty rules
- Timestamp rules
- Transaction validity
- UTXO state transitions
- Block reward
- Monetary issuance

A node must not trust a peer's claimed blockchain height or balance without validation.

Chain selection should use the finalized cumulative-work rules.

---

14. NETWORK RESOURCE MANAGEMENT

Nodes should operate within reasonable hardware limits.

Important resource controls:

- Maximum peer connections
- Maximum message size
- Maximum block size
- Maximum transaction size
- Mempool memory limit
- Network request timeout
- Disk storage monitoring
- Rate limiting

The 16 GB development computer should be sufficient for initial local development, but actual node resource requirements must be measured.

---

15. DATA DIRECTORIES

Each network must use a separate data directory.

Proposed layout:

```
SAECOIN_DATA/
    development/
        blocks/
        chainstate/
        peers/
        logs/

    testnet/
        blocks/
        chainstate/
        peers/
        logs/

    mainnet/
        blocks/
        chainstate/
        peers/
        logs/
```

Wallet data should be stored separately from public node data.

The exact directory names may be adjusted during implementation.

---

16. NETWORK UPGRADE POLICY

Protocol upgrades must be planned and documented.

Each upgrade should specify:

- Upgrade identifier
- Software versions
- Activation conditions
- Compatibility requirements
- Consensus changes
- Testnet results
- Rollback or recovery plan

Consensus changes must never be introduced silently.

A compatible upgrade and a consensus-breaking upgrade must be treated differently.

The activation mechanism remains TO BE DECIDED.

---

17. SECURITY REQUIREMENTS

Network security must address:

- Sybil attacks
- Eclipse attacks
- Peer flooding
- Invalid message flooding
- Oversized message attacks
- Malformed block attacks
- Transaction spam
- Timestamp manipulation
- Difficulty manipulation
- Network partitioning
- Denial-of-service attacks

Peer data must be treated as untrusted.

All incoming data must be validated before being used.

Network message parsing must use strict size limits.

---

18. PARAMETERS REQUIRING FINAL DECISIONS

Before consensus implementation, the following must be finalized:

1. PoW algorithm
2. Hash function
3. Difficulty adjustment algorithm
4. Retarget interval
5. Timestamp rules
6. Maximum block size or weight
7. Transaction size limits
8. Fee policy
9. Network magic bytes
10. Chain identifiers
11. P2P message format
12. Mainnet and testnet genesis blocks
13. Protocol upgrade mechanism
14. Peer connection policy

These are important protocol decisions.

They must not be left to individual developers or AI tools to decide independently.

---

19. FINAL NETWORK PRINCIPLES

1. Mainnet, testnet, and development networks must remain separate.
2. Every node must validate blocks independently.
3. Every consensus rule must be deterministic.
4. Network data must be treated as untrusted.
5. Resource limits must protect ordinary node operators.
6. Difficulty adjustment must be tested against changing hashrate.
7. Monetary issuance must remain within the verified supply rules.
8. Network upgrades must be documented.
9. Consensus-breaking changes require a controlled upgrade process.
10. Decentralization and security must take priority over convenience.

DOCUMENT STATUS

Document: Network_Parameters.md
Version: 1.0
Project: SAECOIN

Consensus: Proof of Work
Transaction Model: UTXO
Status: Draft / Planning
Network Ports: Provisional
Difficulty Algorithm: Not Finalized
Block Size: Not Finalized
Genesis Block: Not Finalized
Implementation Status: Not Started